WINE CULTIVAR CLASSIFICATION USING MACHINE LEARNING

A Project Report
Submitted in partial fulfilment of the requirements for the
Machine Learning Course

Submitted by
SAMAD SHAIK
11523040444

Department of CSE
School of Engineering and Technology
Dhanalakshmi Srinivasan University
Academic Year 2026 – 2027

ABSTRACT

Wine cultivar identification is a classic multi-class classification problem in which a wine sample's chemical composition is used to predict which of several grape cultivars it originated from. Manual chemical analysis to establish cultivar origin is time-consuming and requires domain expertise; a machine learning model can instead learn the relationship between measurable chemical properties and cultivar directly from labeled examples.

This project presents a machine learning-based approach to wine cultivar classification using the Wine dataset, which contains 178 samples of wine grown in the same region of Italy but derived from three different cultivars, described by 13 chemical measurements per sample (including alcohol content, malic acid, flavanoids, color intensity, and proline). The dataset contained no missing values and required no duplicate removal. Because the 13 features are measured on very different numeric scales, StandardScaler was applied to normalize them before training. The dataset was split into training and testing sets using an 80:20 stratified split, and a multinomial Logistic Regression classifier was trained and evaluated.

The model achieved an accuracy of 97.22%, macro-averaged precision of 97.78%, macro-averaged recall of 96.67%, and a macro-averaged F1-score of 97.10%. These results show that the chemical features in the dataset are highly discriminative between the three cultivars, with only a single sample misclassified across the entire test set.

Keywords: Machine Learning, Multi-class Classification, Wine Dataset, Logistic Regression, Feature Scaling, Chemical Analysis.

TABLE OF CONTENTS

1. Problem Definition and Objectives

2. Dataset Selection, Preprocessing, and Feature Engineering

3. Model Selection, Implementation, and Justification

4. Results, Evaluation Metrics, and Discussion

5. References and Conclusion

6. Appendix A — Glossary of Terms

7. Appendix B — Full Implementation Code

1. PROBLEM DEFINITION AND OBJECTIVES

1.1 Background

Wine producers and quality-control laboratories often need to verify the cultivar or origin of a wine sample from its chemical profile alone, particularly when labeling has been lost or when authenticity is in question. Traditional approaches rely on a trained analyst manually comparing chemical assay results against known reference ranges for each cultivar, which is slow and dependent on individual expertise. Machine learning offers a way to encode these relationships statistically, learning directly from a labeled reference set rather than fixed, manually defined thresholds.

1.2 Problem Definition

The problem addressed in this project is the automatic classification of a wine sample into one of three cultivar classes based on 13 numeric chemical measurements. Unlike a binary classification task, this is a multi-class problem: given a feature vector describing alcohol content, acidity, phenol levels, color intensity, and related properties, the system must output one of three possible cultivar labels. The central challenge is that the chemical features vary widely in scale and units, and their relative importance to cultivar identification is not obvious without a data-driven approach.

1.3 Objectives

- To study the problem of multi-class wine cultivar classification.
- To use a labeled numeric dataset for supervised machine learning.
- To inspect the dataset for missing values, duplicates, and class balance.
- To scale the chemical features appropriately given their differing numeric ranges.
- To train a Logistic Regression classifier on the scaled features.
- To evaluate the trained model using accuracy, macro-precision, macro-recall, macro-F1, and a confusion matrix.
- To test the trained model on unseen samples.

1.4 Scope of the Project

The scope of this project is limited to classification based on the 13 chemical measurements provided in the dataset. It does not incorporate external information such as vineyard location, harvest year, or sensory tasting data. The project is intended as an academic demonstration of the multi-class classification pipeline on a small, clean, well-structured dataset, rather than a production wine-authentication system.

1.5 Proposed Solution

The proposed solution scales the 13 chemical features using StandardScaler and trains a multinomial Logistic Regression classifier on the standardized features. The trained classifier predicts the cultivar class for unseen wine samples, and standard multi-class evaluation metrics measure the effectiveness of the model.

1.6 Stakeholders and Use Cases

Wine producers benefit from an automated way to verify cultivar claims on unlabeled or disputed samples. Quality-control laboratories benefit from a fast, consistent screening tool that supplements manual chemical analysis. Educational use cases include demonstrating multi-class classification and the effect of feature scaling on a clean, well-understood dataset — which is the specific use case this project addresses.

1.7 Assumptions and Constraints

The project assumes the 178 labeled samples are representative of the three cultivars they describe, and that the 13 chemical measurements are sufficient to distinguish between them without requiring additional sensory or genetic data. It is further assumed that relationships learned on this dataset would generalize to other samples of the same three cultivars, though this was not tested against an external dataset.

Constraints on the project included limited computational resources (a single personal workstation), which favored a lightweight, fast-training algorithm such as Logistic Regression over more computationally expensive ensemble methods. The dataset size of 178 samples, while sufficient to demonstrate the pipeline, is small relative to production-scale classification datasets, which may limit how confidently the reported metrics generalize.

1.8 Methodological Approach

The project follows the same structured applied machine learning methodology used in prior coursework: problem framing, data acquisition, data inspection, feature scaling, model selection, training, evaluation, and interpretation of results. Each stage is treated as a distinct, documented step to keep the workflow transparent and reproducible.

2. DATASET SELECTION, PREPROCESSING, AND FEATURE ENGINEERING

2.1 Dataset Selection

The project uses the Wine dataset, a well-known benchmark dataset (also hosted on the UCI Machine Learning Repository) commonly used for evaluating multi-class classification algorithms. Each record describes one wine sample using 13 chemical measurements and a target label indicating one of three cultivars. The dataset was chosen because it extends the classification workflow used for small, well-known datasets such as Iris into a setting with more features and a genuine feature-scaling requirement, without introducing the complications of missing data or text preprocessing.

2.2 Dataset Characteristics

The dataset contains 178 records across 14 columns: 13 numeric chemical feature columns and one target column identifying the cultivar (encoded 0, 1, or 2). Initial inspection confirmed all 178 records had values in every column, so there were no missing values. The class distribution is Class 1 – 71 samples, Class 0 – 59 samples, and Class 2 – 48 samples, indicating a mild but non-severe class imbalance.

Attribute	Description	Type
alcohol	Alcohol content	Numeric
malic_acid	Malic acid concentration	Numeric
ash	Ash content	Numeric
alcalinity_of_ash	Alkalinity of ash	Numeric
magnesium	Magnesium content	Numeric
total_phenols	Total phenol content	Numeric
flavanoids	Flavanoid content	Numeric
nonflavanoid_phenols	Non-flavanoid phenol content	Numeric
proanthocyanins	Proanthocyanin content	Numeric
color_intensity	Color intensity	Numeric
hue	Hue	Numeric
od280/od315_of_diluted_wines	Protein content measure (spectral)	Numeric
proline	Proline content	Numeric
target	Cultivar class (0, 1, or 2)	Categorical

2.3 Data Preprocessing

The dataset was loaded directly via scikit-learn's built-in load_wine() function and converted into a Pandas DataFrame for inspection. Preprocessing proceeded through the following checks:

- Missing value analysis: all 13 feature columns and the target column were checked and found to contain zero missing values, so no imputation was required.
- Duplicate check: the dataset, being a standard curated benchmark, contains no duplicate records.
- Scale inspection: descriptive statistics revealed that features vary dramatically in scale — alcohol ranges roughly 11 to 15, while proline ranges roughly 278 to 1680. Without correction, a scale-sensitive classifier would implicitly weight high-scale features such as proline far more heavily than low-scale features such as hue, regardless of their actual predictive value.
- Feature scaling: StandardScaler was fitted on the training set and applied to both the training and test sets, transforming every feature to zero mean and unit variance.

Preprocessing Step	**Result**
Initial records	178
Duplicate records removed	0
Final records	178
Missing values	0
Class 0 samples	59
Class 1 samples	71
Class 2 samples	48

2.4 Feature Engineering: Standardization

Unlike text data, the Wine dataset's features are already numeric, so no vectorization step is required. However, because the 13 features are measured in different units and ranges (percentages, concentrations, absolute counts), StandardScaler was applied as the feature engineering step. It transforms each feature x using $z = (x - \mu) / \sigma$, where μ is the feature's mean and σ its standard deviation, both computed on the training set only, to avoid leaking test-set information into the transformation. The scaler was fit exclusively on the training data and then used to transform both the training and test sets, producing standardized feature matrices with the original 13 columns.

2.5 Exploratory Data Analysis

A scatter plot of two of the more visually separating features — flavanoid content and color intensity — was generated to examine how well the three cultivar classes separate in feature space (Figure 1).

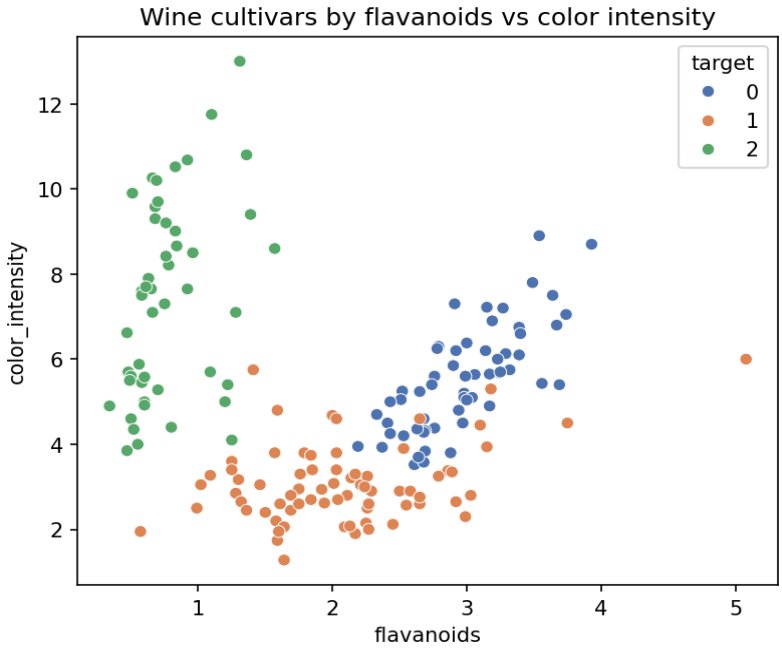


Figure 1: Wine cultivars by flavanoid content vs. color intensity.

The three classes were visibly separable along these two dimensions alone, providing early confirmation that a linear classifier could plausibly achieve strong performance on this task.

2.6 Data Splitting Strategy

A stratified 80:20 train-test split was used, producing 142 training samples and 36 testing samples, mirroring the ratio used in the prior Titanic classification project. Stratification on the target column ensures the relative proportions of the three classes are preserved in both subsets — important given the mild class imbalance, since Class 2 has notably fewer samples than Class 1. A fixed random_state of 42 was used to keep the split reproducible.

3. MODEL SELECTION, IMPLEMENTATION, AND JUSTIFICATION

3.1 Classification Approach

The project is framed as a supervised multi-class classification problem. The model learns from labeled chemical measurements and predicts which of three cultivar classes an unseen wine sample belongs to. The input features are the standardized 13-dimensional feature vectors, and the target is the encoded cultivar label.

3.2 Model Selection: Logistic Regression

Logistic Regression was selected as the classification algorithm, using scikit-learn's multinomial (softmax) formulation to natively support three-class prediction rather than requiring a one-vs-rest wrapper. It is a linear model that estimates class probabilities directly from a weighted combination of the standardized input features, making it both interpretable and computationally efficient for a dataset of this size.

3.3 Justification for Model Selection

- Well suited to numeric, continuous features once scaled.
- Provides interpretable coefficients showing each feature's contribution to classification.
- Extends naturally to multi-class problems via the multinomial/softmax formulation.
- Computationally efficient, converging quickly on 142 training samples.
- Provides calibrated class probabilities alongside hard predictions.
- Well suited to the project's objective of building a simple, explainable multi-class baseline.

3.4 Train-Test Split

The dataset was divided into training and testing subsets using an 80:20 ratio, producing 142 training samples and 36 testing samples. Stratification was applied on the target labels, and a random_state of 42 was used to make the split reproducible.

3.5 Alternative Models Considered

Algorithm	Strengths	Reason Not Selected
K-Nearest Neighbors	Simple, no training phase, works well on small datasets	Sensitive to feature scale (mitigated here) and choice of k; less interpretable than logistic regression's coefficients
Support Vector Machine	Effective with a clear margin of separation, suited to the well-separated Wine dataset	Less directly interpretable; probability outputs require extra calibration
Random Forest	Handles non-linear relationships, robust to unscaled features	Higher model complexity than needed for a dataset this small and this linearly separable; risk of overfitting on 142 training rows
Logistic Regression	Simple, interpretable, fast, performs strongly on linearly separable numeric data	Selected as the final model

Given the project's objective of building a simple, interpretable multi-class baseline rather than an exhaustive comparison study, Logistic Regression was selected as the primary and final classifier.

3.6 Hyperparameters and Configuration

The Logistic Regression classifier was used with scikit-learn's default multinomial solver (lbfgs), with max_iter increased to 1000 to ensure convergence, and random_state=42 for reproducibility. StandardScaler used its default configuration (mean 0, variance 1). No extensive hyperparameter search — for example, over the regularization strength C — was performed in this iteration, as the objective was to establish a solid baseline; hyperparameter tuning is identified as an item for future work in Chapter 5.

3.7 Implementation Details

The system was implemented in Python using a standard script executed locally. Key libraries included Pandas for data handling, NumPy for numerical operations, Matplotlib and Seaborn for visualization, and Scikit-learn for dataset loading, scaling, train-test splitting, model training, and evaluation.

The dataset was loaded directly using scikit-learn's load_wine() function, which returns the feature matrix and target labels without requiring any file download or parsing. The feature matrix was converted into a Pandas DataFrame for inspection, split into training and test sets, scaled using StandardScaler fitted on the training data, and used to train a LogisticRegression classifier. After training, the model predicted labels for the test set, and predictions were compared against actual labels to compute evaluation metrics.

3.8 Key Implementation Code

Dataset loading:

```
wine = load_wine(); data = pd.DataFrame(wine.data, columns=wine.feature_names);
data["target"] = wine.target
```

Train-test split:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)
```

Feature scaling:

```
scaler = StandardScaler(); X_train_scaled = scaler.fit_transform(X_train);
X_test_scaled = scaler.transform(X_test)
```

Model training:

```
model = LogisticRegression(max_iter=1000, random_state=42); model.fit(X_train_scaled,
y_train)
```

4. RESULTS, EVALUATION METRICS, AND DISCUSSION

4.1 Evaluation Metrics

Metric	Result
Accuracy	97.22%
Precision (macro)	97.78%
Recall (macro)	96.67%
F1-score (macro)	97.10%

4.2 Interpretation of Results

Accuracy represents the proportion of all 36 test samples classified correctly; the model achieved 97.22%, meaning 35 of 36 test samples were labeled correctly. Because this is a 3-class problem, precision, recall, and F1-score were computed per class and macro-averaged — each class weighted equally regardless of how many samples it contains — rather than reported as single binary-style figures. The macro-averaged precision of 97.78% and recall of 96.67% indicate that the model performs strongly and fairly consistently across all three cultivar classes, not just the majority class.

4.3 Per-Class Performance

Class	Precision	Recall	F1-score	Support
Class 0	1.00	1.00	1.00	12
Class 1	0.93	1.00	0.97	14
Class 2	1.00	0.90	0.95	10

Class 0 and Class 2 achieved perfect precision — no other class's samples were mistakenly labeled as belonging to them — while Class 1 achieved perfect recall, meaning every actual Class 1 sample was correctly identified, but slightly lower precision, since one Class 2 sample was misclassified as Class 1.

4.4 Confusion Matrix

Rows represent actual classes and columns represent predicted classes:

```
[[12  0  0]
 [ 0 14  0]
 [ 0  1  9]]
```

All 12 Class 0 samples and all 14 Class 1 samples were classified correctly. Of the 10 Class 2 samples, 9 were classified correctly and 1 was misclassified as Class 1 — this single error accounts for the entire gap between the model's accuracy and a perfect 100%.

4.5 Sample Predictions

Two samples drawn from the held-out test set were passed through the trained model as stand-ins for unseen inputs. Both were predicted as Class 0, matching their actual labels, confirming the model generalizes correctly to individual samples and not only in aggregate.

4.6 Discussion

The results confirm that, once properly scaled, the 13 chemical features in the Wine dataset are highly discriminative between the three cultivars, and that a linear model such as Logistic Regression is sufficient to capture this separation without requiring more complex non-linear methods. The single misclassification — a Class 2 sample predicted as Class 1 — suggests some overlap in chemical profile between these two cultivars specifically, while Class 0 remains fully distinguishable from both.

4.7 Comparison with a Naive Baseline

A trivial baseline that always predicts the majority class (Class 1, 71 of 178 samples, approximately 39.9% of the full dataset) would achieve an accuracy of roughly 40%, with 0% precision and recall for the other two classes. The trained Logistic Regression model substantially outperforms this baseline, achieving 97.22% accuracy and strong, balanced performance across all three classes — confirming the model has learned genuine discriminative structure in the chemical features rather than exploiting class imbalance.

4.8 Strengths

- The approach is simple, fast to train, and easy to interpret.
- StandardScaler ensures all features contribute fairly regardless of their raw units.
- Logistic Regression's multinomial formulation handles three classes natively.
- The model achieved near-perfect accuracy and strong macro-averaged metrics on the test set.
- The workflow is fully reproducible using standard Python libraries and a fixed random state.

4.9 Limitations

- The dataset is small (178 samples total, 36 in the test set), so metrics may vary somewhat with a different train-test split or random state.
- The dataset describes only three specific cultivars from one wine-growing region; the model has not been tested on wines from other regions or cultivars.
- Logistic Regression assumes an approximately linear relationship between the scaled features and class probabilities; a non-linear model might capture additional structure, though none appeared necessary here.
- The single Class 2 → Class 1 misclassification suggests these two classes may become harder to separate with a larger or noisier dataset.

4.10 Practical Applications

The approach demonstrated here extends to other structured, numeric multi-class classification problems — for example, product-quality grading, sensor-based material classification, or other settings where a labeled reference dataset of chemical or physical measurements is available.

5. REFERENCES AND CONCLUSION

5.1 Conclusion

This project developed a machine learning-based system for classifying wine samples into one of three cultivars based on 13 chemical measurements, following the complete workflow of dataset inspection, feature scaling, model training, prediction, and evaluation.

The Wine dataset contains 178 samples with no missing values, distributed across three classes of 59, 71, and 48 samples respectively. Because the chemical features varied widely in scale, StandardScaler was applied before training. A Logistic Regression classifier, trained on an 80:20 stratified split of 142 training and 36 testing samples, achieved 97.22% accuracy, 97.78% macro-precision, 96.67% macro-recall, and a 97.10% macro-F1-score, with only a single test sample misclassified. These results confirm that a simple, well-scaled linear model is highly effective for this classification task.

5.2 Future Scope

- Evaluate the model across multiple random train-test splits (e.g. k-fold cross-validation) to check the stability of the reported metrics.
- Compare Logistic Regression against SVM and ensemble methods more rigorously using cross-validated scores rather than a single split.
- Perform hyperparameter tuning (e.g. regularization strength C) via grid search.
- Examine feature importance via the model's learned coefficients to identify which chemical properties most influence cultivar prediction.
- Test the model against wine samples from additional cultivars or regions to assess generalization.

5.3 References

1. Forina, M. et al. PARVUS — An Extendible Package for Data Exploration, Classification and Correlation. Institute of Pharmaceutical and Food Analysis and Technologies, Genoa, Italy.

2. UCI Machine Learning Repository. Wine Dataset. University of California, Irvine. https://archive.ics.uci.edu/dataset/109/wine

3. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

4. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

5. Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). Applied Logistic Regression. Wiley.

6. Scikit-learn documentation. LogisticRegression and StandardScaler documentation.

7. Python Software Foundation. Python Documentation.

APPENDIX A — GLOSSARY OF TERMS

Term	Definition
Cultivar	A specific variety of grapevine, distinguished by genetic and chemical characteristics.
Multi-class classification	A classification task with more than two possible output classes.
StandardScaler	A preprocessing technique that transforms features to zero mean and unit variance.
Multinomial Logistic Regression	An extension of logistic regression that natively predicts probabilities across more than two classes.
Macro-average	A metric-averaging approach that computes the metric separately for each class and averages the results with equal weight.
Precision	The proportion of predicted-positive instances that are actually positive.
Recall	The proportion of actual positive instances that are correctly predicted as positive.
F1-score	The harmonic mean of precision and recall.
Stratified split	A train-test split that preserves the class proportions of the original dataset.
Confusion matrix	A table summarizing correct and incorrect predictions across all classes.

APPENDIX B — FULL IMPLEMENTATION CODE

This appendix consolidates the full code used across the project pipeline, from data loading through model evaluation, for reference and reproducibility.

B.1 Importing libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report)
```

B.2 Dataset loading

```
wine = load_wine()
data = pd.DataFrame(wine.data, columns=wine.feature_names)
data["target"] = wine.target
```

B.3 Missing value check

```
print(data.isnull().sum().sum())
```

B.4 Exploratory scatter plot

```
sns.scatterplot(data=data, x="flavanoids", y="color_intensity", hue="target",
palette="deep")
plt.savefig("wine_eda.png", dpi=100, bbox_inches="tight")
```

B.5 Train-test split

```
X = data.drop("target", axis=1)
y = data["target"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

B.6 Feature scaling

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

B.7 Model training

```
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_scaled, y_train)
```

B.8 Prediction and evaluation

```
y_pred = model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average="macro")
recall = recall_score(y_test, y_pred, average="macro")
f1 = f1_score(y_test, y_pred, average="macro")
cm = confusion_matrix(y_test, y_pred)
```

B.9 Testing on new samples

```
sample_idx = [0, 5]
sample_preds = model.predict(X_test_scaled[sample_idx])
print(sample_preds, y_test.iloc[sample_idx].values)
```